

Activity Log Analyzer in Python

Python Code

```
from typing import List, Dict, Set
from collections import defaultdict
from functools import reduce

def total_time_per_user(logs: List[Dict]) -> Dict[str, float]:
    totals = defaultdict(float)

    def reducer(acc, log):
        acc[log["user"]] += log["duration"]
        return acc

    return dict(reduce(reducer, logs, totals))

def most_active_users(logs: List[Dict], k: int) -> List[str]:
    totals = total_time_per_user(logs)

    return [
        user for user, _ in
        sorted(
            totals.items(),
            key=lambda item: item[1],
            reverse=True
        )[:k]
    ]

def unique_actions(logs: List[Dict]) -> Set[str]:
    return {log["action"] for log in logs}

logs = [
    {"user": "220101", "action": "YouTube", "duration": 2.5},
    {"user": "220102", "action": "Instagram", "duration": 1.5},
    {"user": "220101", "action": "WhatsApp", "duration": 1.0},
    {"user": "220103", "action": "Chrome", "duration": 3.0},
    {"user": "220102", "action": "YouTube", "duration": 2.0}
]

print("Total Time Per User:")
print(total_time_per_user(logs))

print("\nMost Active Users:")
print(most_active_users(logs, 2))

print("\nUnique Actions:")
print(unique_actions(logs))
```

Complexity Analysis

a. Time Complexity for Computing Top K Users:

Computing total activity time requires traversing all logs once: $O(n)$. Sorting users by total activity time requires $O(u \log u)$, where u is the number of unique users. Therefore, the overall complexity is $O(n + u \log u)$.

b. Space Complexity of Storing Intermediate Results:

The dictionary storing total time per user requires $O(u)$ space, where u is the number of unique users. The set of unique actions requires $O(a)$ space, where a is the number of distinct actions. Overall auxiliary space complexity is $O(u + a)$.